WEEK 5: KRUSKAL'S ALGORITHM

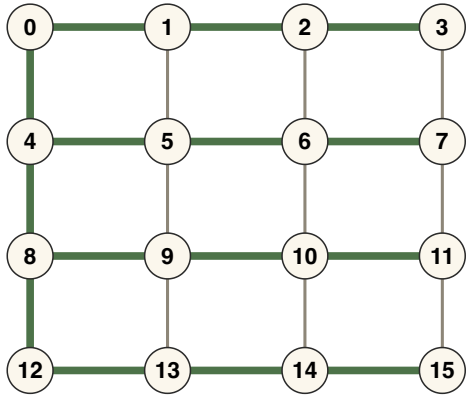
Advanced Algorithms

ROADMAP

1. **Cuts and Safe Edges**
2. **Kruskal's Algorithm**
3. **The Graphic Matroid** — a second proof
4. **Union-Find**

SPANNING TREES

Definition A **spanning tree** of a connected graph $G = (V, E)$ is a subgraph of G that is a tree and whose vertex set is V — a tree that connects all the vertices of G.



A spanning tree (green) of the 4×4 grid graph.

DFS or BFS finds *a* spanning tree in time $O(|E|)$ — this one is from BFS at vertex 0.

MINIMUM SPANNING TREE

Today we have a different goal in finding a spanning tree.

> **Problem** Find a spanning tree of G where the **sum of the weights** of the edges in the tree is as small as possible: a **minimum spanning tree (MST)**.

MST is one of the oldest problems in combinatorial optimisation: the first algorithm was given in 1926 by Borůvka, to plan an electrical network for Moravia.

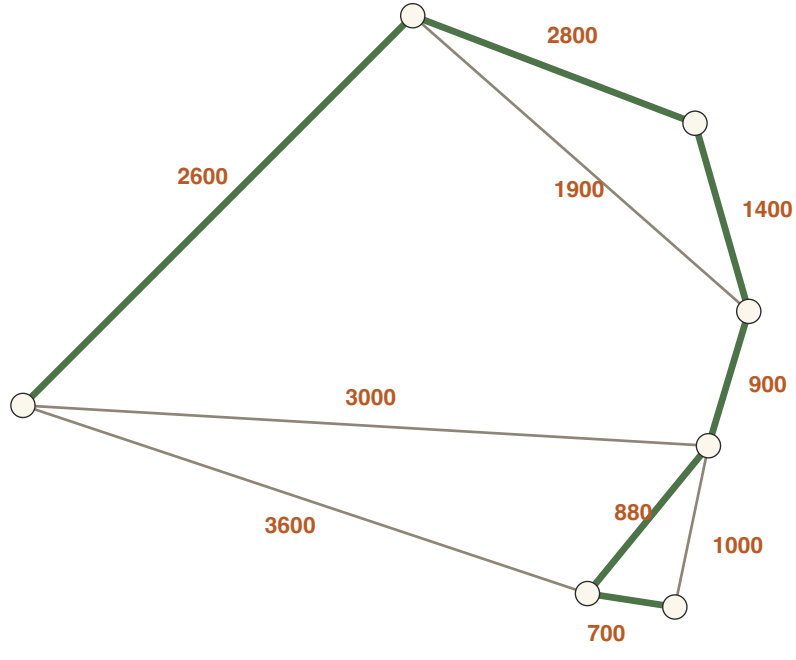
This week: Kruskal's algorithm (1956). Next week: Prim's algorithm — and Borůvka's original one.

EXAMPLE: LINKING CITIES BY CABLE

Vertices are cities. Edges show where cable can be laid, weighted by distance (km).

Goal: link all cities with the least total cable. That is exactly a minimum spanning tree.

Here the MST (green) needs 9280 km of cable.



Ten candidate cable routes; the six green ones form the MST.

WARM-UP: UNWEIGHTED GRAPHS

Suppose all edge weights are one. What is the total weight of an MST in an unweighted n-vertex connected graph?

Every spanning tree has exactly $n - 1$ edges, so *any* spanning tree is a minimum spanning tree, and has weight $n - 1$.

So the unweighted case is solved by depth- or breadth-first search. The interesting case is general weights.

ASSUMPTION: UNIQUE WEIGHTS

We will assume that no two edge weights compare as equal.

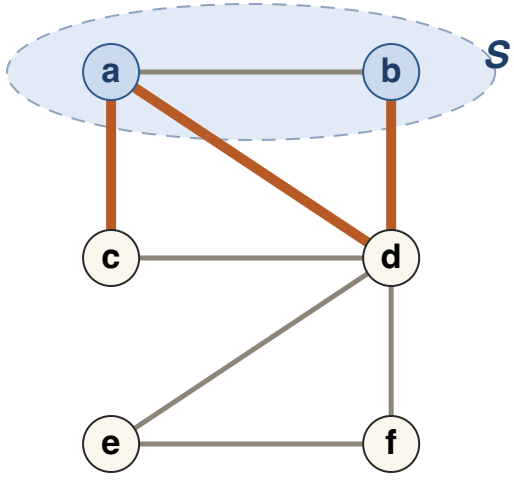
Easy to ensure: break ties between equal-weight edges by lexicographic order of endpoint names.

> **FACT** If no two edge weights compare as equal, the minimum spanning tree is **unique**.

This keeps statements like "the MST must contain edge e" clean.

CUTS AND SAFE EDGES

CUTS



$S = \{a, b\}$: the cut has three edges (orange).

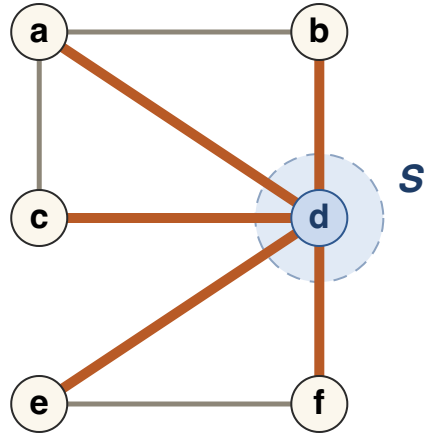
Cuts are the key to thinking about minimum spanning trees.

Definition A **cut** is defined by a non-empty subset of vertices $S \subset V, S \neq V$. The cut is the set of edges with **exactly one endpoint in** S.

Here the graph has edges $ab, ac, ad, bd, cd, de, ef, df$ — the same example graph we use on the next slides.

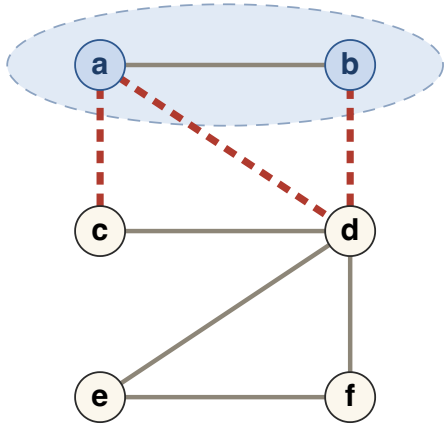
CUTS

A cut can also be defined by a single vertex.

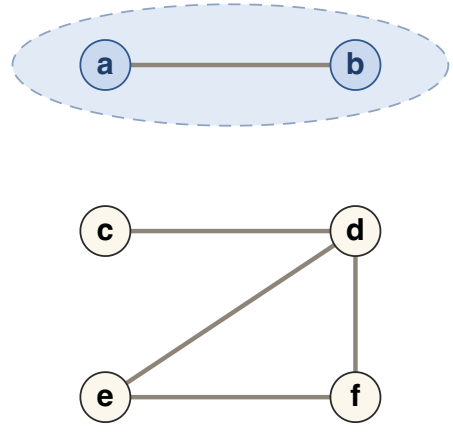


$S = \{d\}$: all five edges at d are cut edges.

REMOVING A CUT DISCONNECTS THE GRAPH



Delete the cut edges (dashed red) …



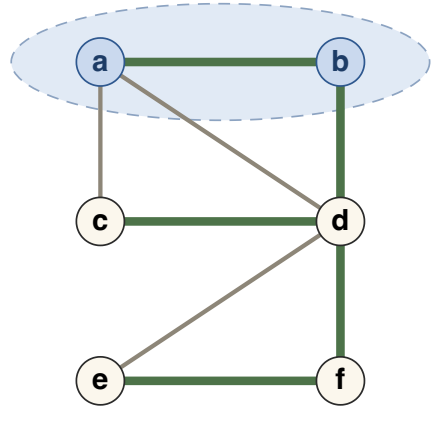
… and no edge connects S to the rest.

Removing the edges of any cut disconnects the graph: there is no longer any edge between S and $V \setminus S$.

FACT 1: SPANNING TREES MEET EVERY CUT

FACT 1 A spanning tree must contain at least one edge from every cut.

If there were a cut from which the tree contained no edge, removing that cut would not touch the tree — yet it separates S from $V \setminus S$. Then the tree could not be connected. ■



This spanning tree (green) crosses the cut of $S = \{a, b\}$ via edge bd.

FACT 2: THE CUT RULE

> **FACT 1** A spanning tree must contain an edge from every cut.

For a *minimum* spanning tree we can say more.

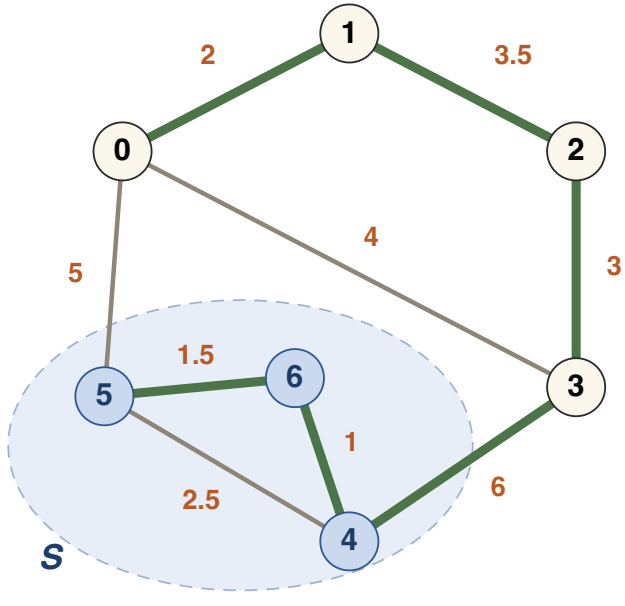
> **FACT 2** A minimum spanning tree must contain the **minimum-weight edge** from every cut.

Fact 2 is the basis of every minimum spanning tree algorithm — Kruskal, Prim, and Borůvka all follow from it.

PROOF OF FACT 2 (1/3)

Take any cut S; here $S = \{4, 5, 6\}$, with cut edges $\{0, 5\}$ of weight 5 and $\{3, 4\}$ of weight 6.

By Fact 1 a spanning tree contains at least one cut edge. Suppose, for contradiction, a spanning tree T (green) does *not* contain the minimum-weight cut edge $\{0, 5\}$ — it crosses the cut only via $\{3, 4\}$.

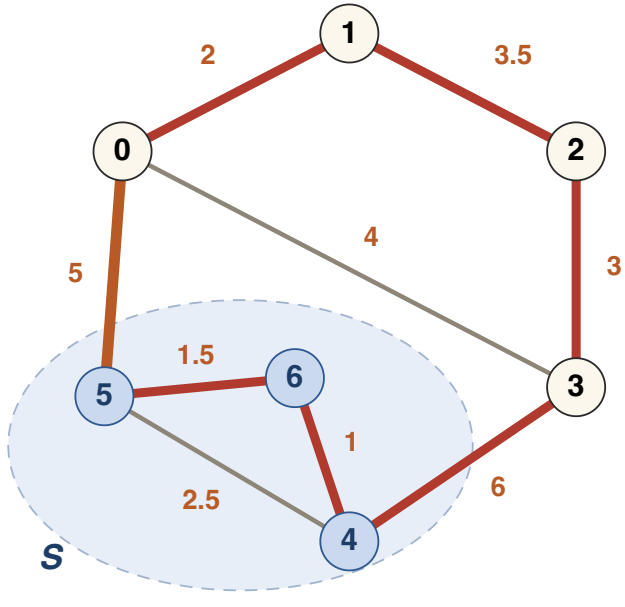


Tree T in green; cut $S = \{4, 5, 6\}$; min-weight cut edge $\{0, 5\}$ not in T.

PROOF OF FACT 2 (2/3)

Add the minimum-weight cut edge $\{0, 5\}$ (orange) to T. This creates a cycle: $0-1-2-3-4-6-5-0$.

The cycle crosses the cut an even number of times, so it contains *another* cut edge besides $\{0, 5\}$ — here $\{3, 4\}$ — and that edge is heavier (weight $6 > 5$).

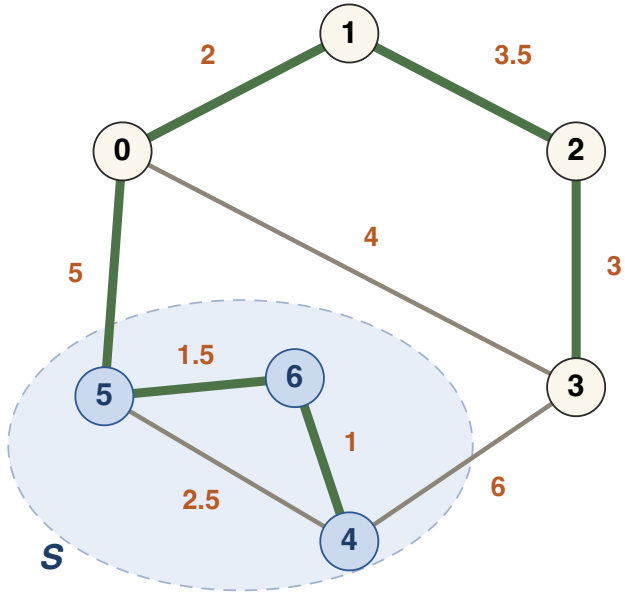


The cycle (red) must cross the cut again — swap out the heavier crossing edge.

PROOF OF FACT 2 (3/3)

Swap: remove $\{3, 4\}$ and keep $\{0, 5\}$ in its place. The result is still a spanning tree, and it is lighter by $6 - 5 = 1$.

So T was not minimum after all — a contradiction. A minimum spanning tree must contain the minimum-weight edge of every cut. ■



The new tree (green): $\{0, 5\}$ in, $\{3, 4\}$ out — total weight $17 \to 16$.

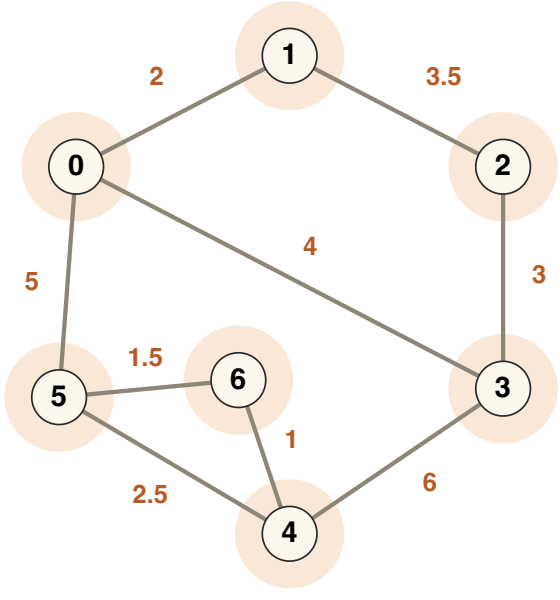
KRUSKAL'S ALGORITHM

KRUSKAL: THE PLAN

- Maintain a **partition** of the vertices into sets $S_1, \ldots, S_k$, starting with n singletons, and a set A of chosen edges, starting empty.
- A always forms a tree on each set S_i — a forest — and every edge of A is part of the MST.
- Go over the edges **from lightest to heaviest**, taking an edge exactly when its endpoints lie in *different* sets — then merge those two sets.

Sets merge only when we add an edge, so the partition is always exactly the connected components of A.

WORKED EXAMPLE: SETUP



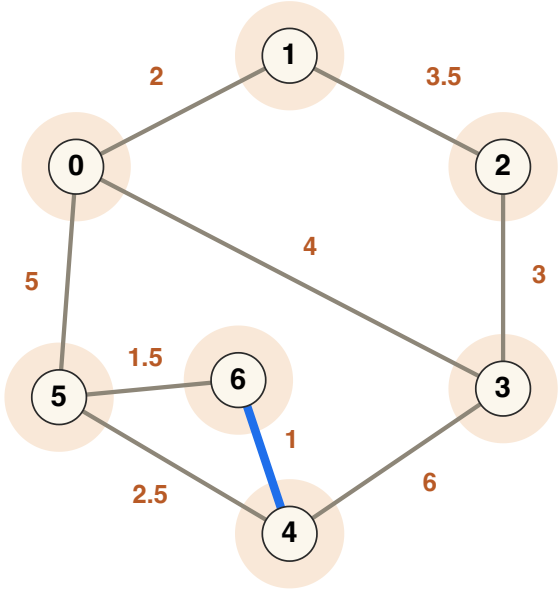
Each vertex starts in its own set of the partition.

Initially every set of the partition is a single vertex: $\{\{0\}, \{1\}, \ldots, \{6\}\}$, and $A = \emptyset$.

Kruskal walks the edges in sorted order, shown along the bottom of the next slides. The edge under consideration is blue — in the list and in the picture; faded edges are already dealt with.

$$\{4,6\}_1 \prec \{5,6\}_{1.5} \prec \{0,1\}_2 \prec \{4,5\}_{2.5} \prec \{2,3\}_3 \prec \{1,2\}_{3.5} \prec \{0,3\}_4 \prec \{0,5\}_5 \prec \{3,4\}_6$$

ROUND 1: EDGE $\{4, 6\}$, WEIGHT 1

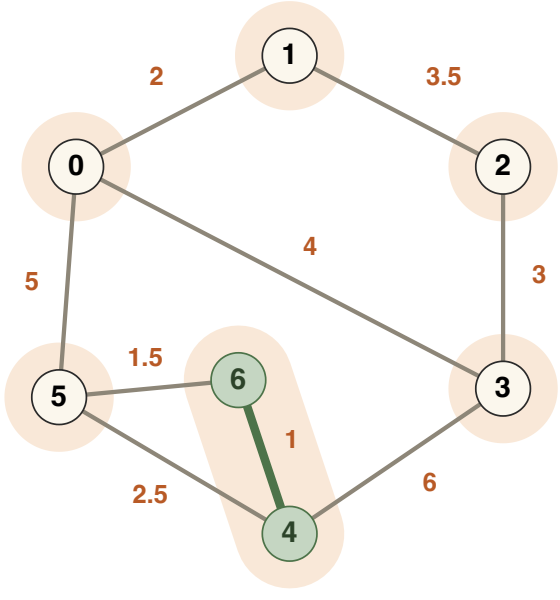


The lightest edge in the whole graph is $\{4, 6\}$ (blue). Its endpoints lie in different sets of the partition.

It is the minimum-weight edge in the cut defined by vertex 4 (or by vertex 6). By Fact 2 it must be in the MST.

$$\{4,6\}_1 \prec \{5,6\}_{1.5} \prec \{0,1\}_2 \prec \{4,5\}_{2.5} \prec \{2,3\}_3 \prec \{1,2\}_{3.5} \prec \{0,3\}_4 \prec \{0,5\}_5 \prec \{3,4\}_6$$

ROUND 1: EDGE $\{4, 6\}$, WEIGHT 1

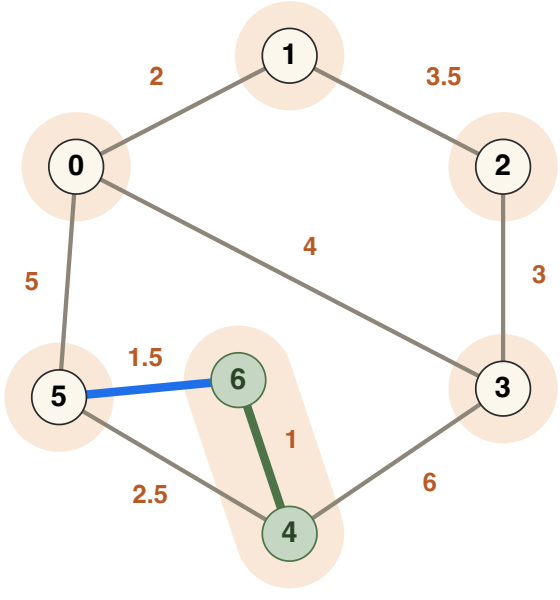


Add $\{4, 6\}$ to A and merge the sets $\{4\}$ and $\{6\}$.

Now $A = \{\{4, 6\}\}$ and the partition is $\{\{0\}, \{1\}, \{2\}, \{3\}, \{5\}, \{4, 6\}\}$.

$$\{4,6\}_1 \prec \{5,6\}_{1.5} \prec \{0,1\}_2 \prec \{4,5\}_{2.5} \prec \{2,3\}_3 \prec \{1,2\}_{3.5} \prec \{0,3\}_4 \prec \{0,5\}_5 \prec \{3,4\}_6$$

ROUND 2: EDGE $\{5, 6\}$, WEIGHT 1.5

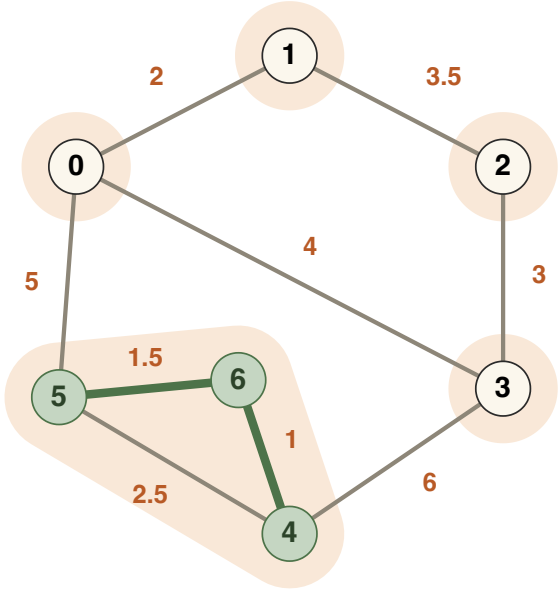


The next lightest edge, $\{5, 6\}$, has endpoints in *distinct* sets: $\{5\}$ and $\{4, 6\}$.

It is the minimum-weight edge in the cut defined by $\{5\}$ — or the cut defined by $\{4, 6\}$. By Fact 2 it is in the MST.

$$\{4,6\}_1 \prec \{5,6\}_{1.5} \prec \{0,1\}_2 \prec \{4,5\}_{2.5} \prec \{2,3\}_3 \prec \{1,2\}_{3.5} \prec \{0,3\}_4 \prec \{0,5\}_5 \prec \{3,4\}_6$$

ROUND 2: EDGE $\{5, 6\}$, WEIGHT 1.5

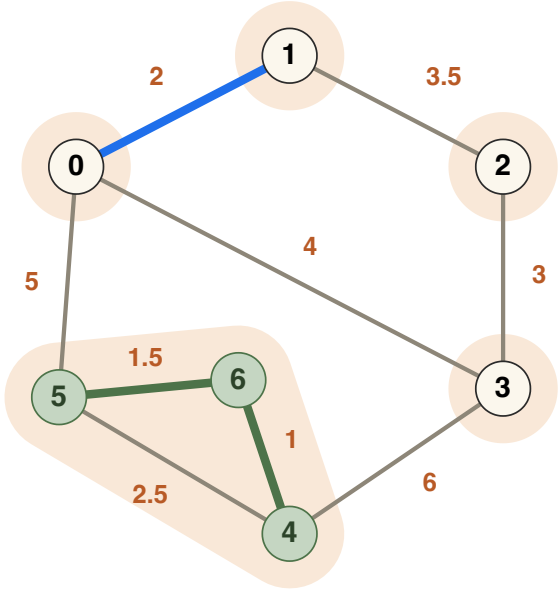


Add $\{5, 6\}$ to A and merge $\{5\}$ with $\{4, 6\}$.

$A = \{\{4, 6\}, \{5, 6\}\}$; partition $\{\{0\}, \{1\}, \{2\}, \{3\}, \{4, 5, 6\}\}$.

$$\{4,6\}_1 \prec \{5,6\}_{1.5} \prec \{0,1\}_2 \prec \{4,5\}_{2.5} \prec \{2,3\}_3 \prec \{1,2\}_{3.5} \prec \{0,3\}_4 \prec \{0,5\}_5 \prec \{3,4\}_6$$

ROUND 3: EDGE $\{0, 1\}$, WEIGHT 2

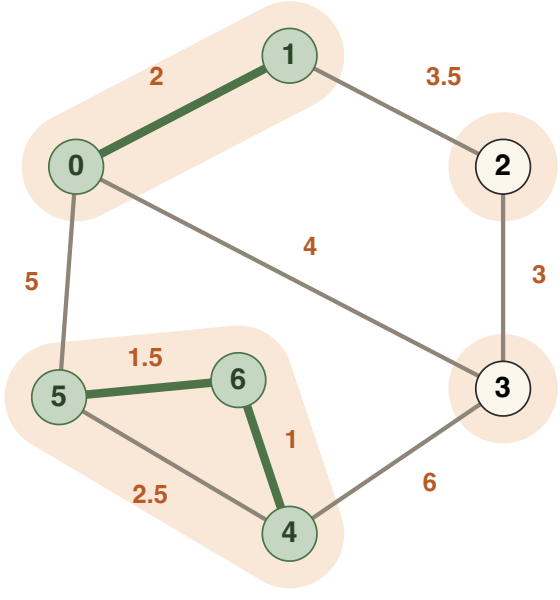


$\{0, 1\}$ (weight 2) joins two singletons — its endpoints are in distinct sets.

It is the lightest edge across the cut defined by $\{0\}$, so it is in the MST.

$$\{4,6\}_1 \prec \{5,6\}_{1.5} \prec \{0,1\}_2 \prec \{4,5\}_{2.5} \prec \{2,3\}_3 \prec \{1,2\}_{3.5} \prec \{0,3\}_4 \prec \{0,5\}_5 \prec \{3,4\}_6$$

ROUND 3: EDGE $\{0, 1\}$, WEIGHT 2

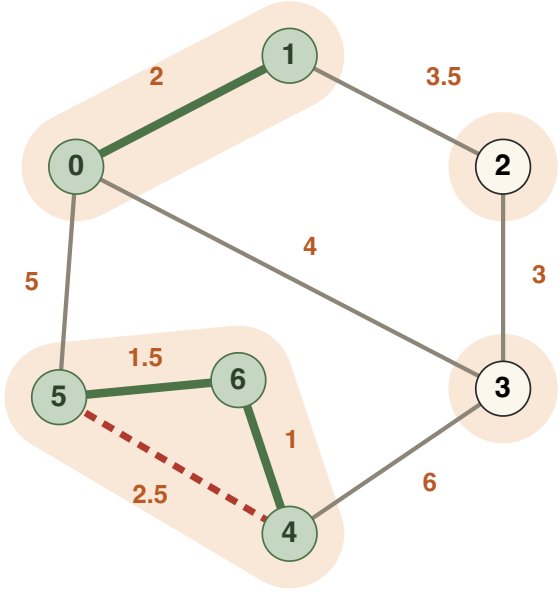


Add $\{0, 1\}$ and merge $\{0\}$ with $\{1\}$.

Partition: $\{\{0, 1\}, \{2\}, \{3\}, \{4, 5, 6\}\}$.

$\{4, 6\}_1 \prec \{5, 6\}_{1.5} \prec \{0, 1\}_2 \prec \{4, 5\}_{2.5} \prec \{2, 3\}_3 \prec \{1, 2\}_{3.5} \prec \{0, 3\}_4 \prec \{0, 5\}_5 \prec \{3, 4\}_6$

ROUND 4: EDGE $\{4, 5\}$, WEIGHT 2.5

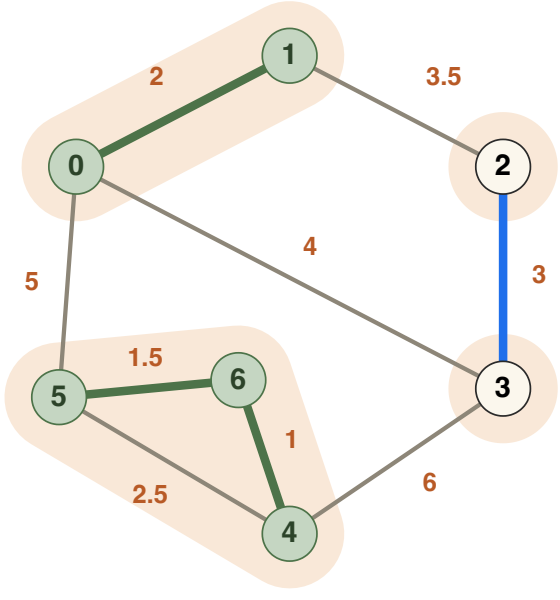


Something different happens: $\{4, 5\}$ (weight 2.5, dashed red) has *both endpoints in the same set*. Adding it would create the cycle $4-5-6-4$.

This edge cannot be part of the MST. We take no action: A and the partition are unchanged.

$$\{4,6\}_1 \prec \{5,6\}_{1.5} \prec \{0,1\}_2 \prec \{4,5\}_{2.5} \prec \{2,3\}_3 \prec \{1,2\}_{3.5} \prec \{0,3\}_4 \prec \{0,5\}_5 \prec \{3,4\}_6$$

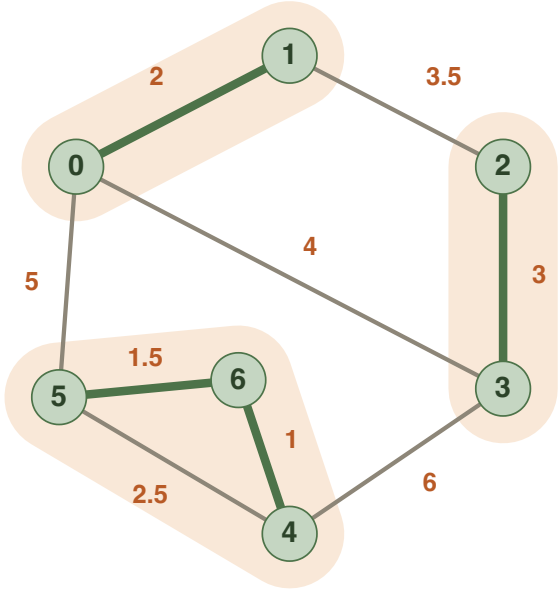
ROUND 5: EDGE $\{2, 3\}$, WEIGHT 3



$\{2, 3\}$ (weight 3) joins two singletons — the lightest edge across the cut defined by $\{2\}$, so it is in the MST.

$$\{4,6\}_1 \prec \{5,6\}_{1.5} \prec \{0,1\}_2 \prec \{4,5\}_{2.5} \prec \{2,3\}_3 \prec \{1,2\}_{3.5} \prec \{0,3\}_4 \prec \{0,5\}_5 \prec \{3,4\}_6$$

ROUND 5: EDGE $\{2, 3\}$, WEIGHT 3

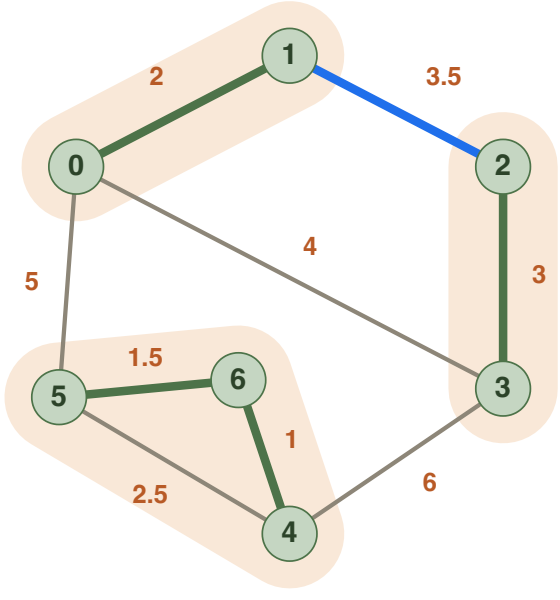


Add $\{2, 3\}$ and merge $\{2\}$ with $\{3\}$.

Partition: $\{\{0, 1\}, \{2, 3\}, \{4, 5, 6\}\}$.

$\{4, 6\}_1 \prec \{5, 6\}_{1.5} \prec \{0, 1\}_2 \prec \{4, 5\}_{2.5} \prec \{2, 3\}_3 \prec \{1, 2\}_{3.5} \prec \{0, 3\}_4 \prec \{0, 5\}_5 \prec \{3, 4\}_6$

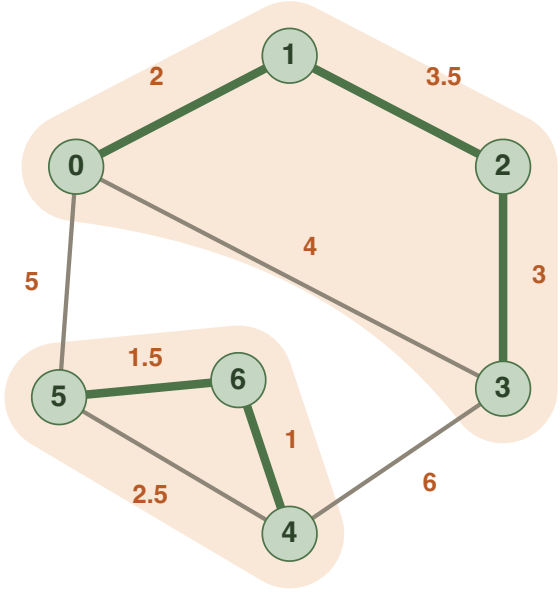
ROUND 6: EDGE $\{1, 2\}$, WEIGHT 3.5



$\{1, 2\}$ (weight 3.5) has endpoints in $\{0, 1\}$ and $\{2, 3\}$ — the lightest edge between distinct sets, so it is in the MST.

$$\{4,6\}_1 \prec \{5,6\}_{1.5} \prec \{0,1\}_2 \prec \{4,5\}_{2.5} \prec \{2,3\}_3 \prec \{1,2\}_{3.5} \prec \{0,3\}_4 \prec \{0,5\}_5 \prec \{3,4\}_6$$

ROUND 6: EDGE $\{1, 2\}$, WEIGHT 3.5

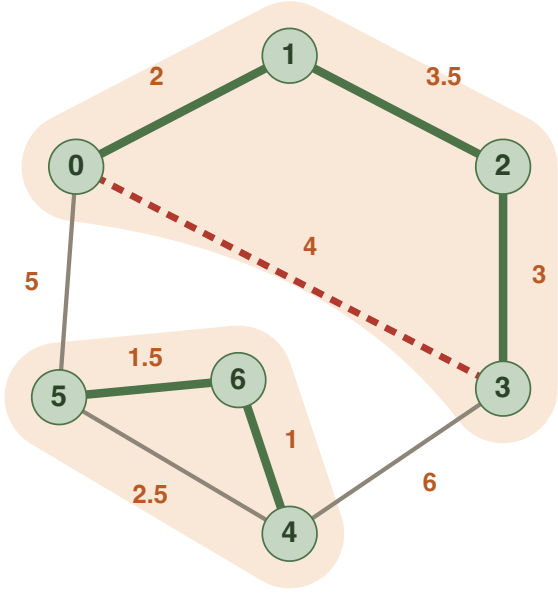


Add $\{1, 2\}$ and merge $\{0, 1\}$ with $\{2, 3\}$.

$A = \{\{4, 6\}, \{5, 6\}, \{0, 1\}, \{2, 3\}, \{1, 2\}\}$;
partition $\{\{0, 1, 2, 3\}, \{4, 5, 6\}\}$.

$$\{4,6\}_1 \prec \{5,6\}_{1.5} \prec \{0,1\}_2 \prec \{4,5\}_{2.5} \prec \{2,3\}_3 \prec \{1,2\}_{3.5} \prec \{0,3\}_4 \prec \{0,5\}_5 \prec \{3,4\}_6$$

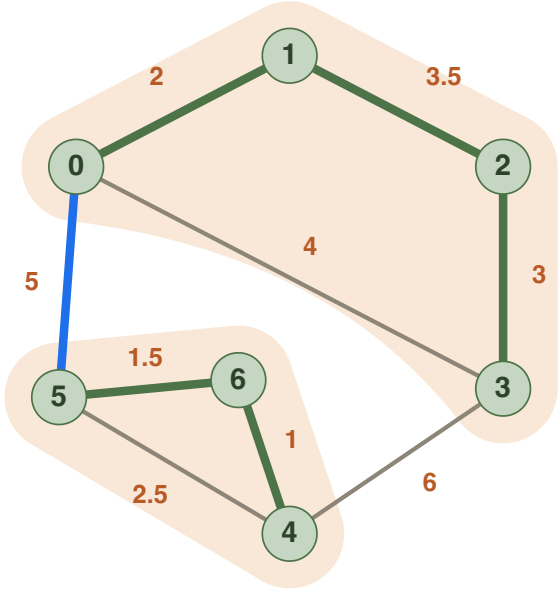
ROUND 7: EDGE $\{0, 3\}$, WEIGHT 4



$\{0, 3\}$ (weight 4, dashed red) has both endpoints in $\{0, 1, 2, 3\}$ — it would create a cycle. Skip.

$$\{4,6\}_1 \prec \{5,6\}_{1.5} \prec \{0,1\}_2 \prec \{4,5\}_{2.5} \prec \{2,3\}_3 \prec \{1,2\}_{3.5} \prec \{0,3\}_4 \prec \{0,5\}_5 \prec \{3,4\}_6$$

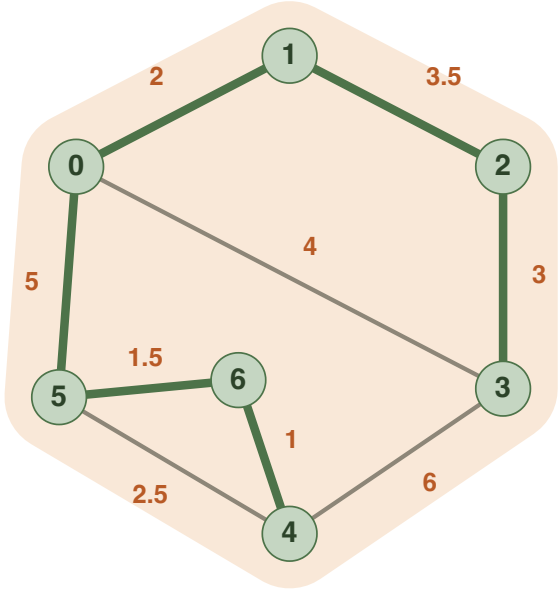
ROUND 8: EDGE $\{0, 5\}$, WEIGHT 5



$\{0, 5\}$ (weight 5) is the lightest edge joining $\{0, 1, 2, 3\}$ and $\{4, 5, 6\}$ — by Fact 2 it is in the MST.

$$\{4,6\}_1 \prec \{5,6\}_{1.5} \prec \{0,1\}_2 \prec \{4,5\}_{2.5} \prec \{2,3\}_3 \prec \{1,2\}_{3.5} \prec \{0,3\}_4 \prec \{0,5\}_5 \prec \{3,4\}_6$$

ROUND 8: EDGE $\{0, 5\}$, WEIGHT 5



Add $\{0, 5\}$ and merge. The partition is now one set: Kruskal is done!

The MST has weight $1 + 1.5 + 2 + 3 + 3.5 + 5 = 16$; edge $\{3, 4\}$ is never needed.

$$\{4, 6\}_1 \prec \{5, 6\}_{1.5} \prec \{0, 1\}_2 \prec \{4, 5\}_{2.5} \prec \{2, 3\}_3 \prec \{1, 2\}_{3.5} \prec \{0, 3\}_4 \prec \{0, 5\}_5 \prec \{3, 4\}_6$$

KRUSKAL: SUMMARY

INITIALISATION Partition the vertices into n singleton sets. The forest-edge set A starts empty.

INVARIANT A partition of the vertices into sets $S_1, \ldots, S_k$; the edges A form a tree on each S_i, and every edge of A is in the MST.

ROUND i Take the i^{th} lightest edge e. Endpoints in the same set: do nothing. Otherwise $A \leftarrow A \cup \{e\}$ and merge the two sets.

WHY IT WORKS

Every added edge is safe. In round i, if e has endpoints in distinct sets of the partition, it must be the lightest edge with this property:

- every lighter edge either merged two sets earlier, or already had both endpoints in one set;
- so e is the minimum-weight edge in the cut defined by the set containing either endpoint — by Fact 2 it belongs to the MST.

So the invariant holds after every round: A is a forest, a tree on each set of the partition, and $A \subseteq$ MST.

TERMINATION

Suppose the algorithm terminates with at least two sets S_1, S_2 remaining in the partition.

- Then the algorithm considered *all* the edges of the graph and never found an edge in the cut defined by S_1.
- This implies the graph is not connected — a contradiction.

So the partition ends as a single set.

THE OUTPUT IS AN MST

A ends with $n - 1$ safe edges forming a spanning tree, and every edge of A is in the MST. So A *is* the minimum spanning tree.

> **CAREFUL** On a *disconnected* graph Kruskal ends with several sets — it computes the minimum spanning **forest**.

THE GRAPHIC MATROID

A second proof

A SECOND PROOF

The cut rule shows why Kruskal's algorithm is correct — but it is an argument about graphs and cuts, tailored to this one problem.

Now we place the algorithm in a more general context: *when do greedy algorithms succeed?* That context is **matroids**.

Greedy is optimal over any matroid — so it is enough to show that the forests of G form one.

FORESTS

Definition A set of edges that does not contain a cycle is called a **forest**. A forest is a collection of trees.

Let G be a connected, weighted, undirected graph. Call a subset of edges of G **admissible** if it is a forest.

The sets A built by Kruskal are exactly admissible sets. Let's check the axioms.

THE INDEPENDENCE AXIOMS

Admissible sets = forests of G. They satisfy:

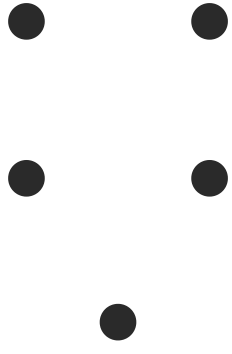
P0 The empty set is admissible.

P1 (HEREDITARY) Any subset of an admissible set is admissible. (Removing edges cannot create a cycle.)

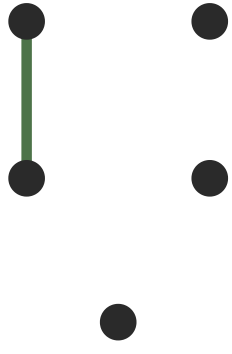
P2 (EXCHANGE PROPERTY) If A and B are admissible sets with $|A| < |B|$, then there is an $e \in B \setminus A$ such that $A \cup \{e\}$ is admissible.

WHY THE EXCHANGE PROPERTY HOLDS

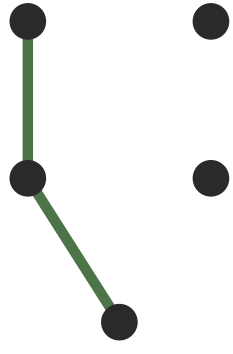
Key observation: every time we add an edge that does not create a cycle, the number of connected components goes down by one.



5 components



4 components



3 components

So a forest with k edges on n vertices has exactly $n - k$ connected components.

EXCHANGE PROPERTY: PROOF

Let A, B be forests with $|A| < |B|$.

- Since components $= n - |\text{edges}|$: B has *fewer* connected components than A.
- So there must be vertices u, v lying in *different* components of A with $\{u, v\} \in B$. (If every B-edge stayed inside one A-component, each B-component would sit inside an A-component, giving B at least as many components.)
- Adding $\{u, v\}$ to A connects two different components, so it does not create a cycle.
- Hence there is $e \in B \setminus A$ with $A \cup \{e\}$ admissible. ■

THE GRAPHIC MATROID

Definition The forests of a graph satisfy P0, P1, P2: they form a **matroid**, the **graphic matroid**.

Theorem In any matroid, the **generic greedy algorithm** finds a maximum-size admissible set of optimal weight.

```
// generic greedy in a matroid (minimisation)
sort the elements by weight, smallest first
A ← ∅
for each element e in sorted order:
    if A ∪ {e} is admissible:   // adding e keeps A a forest
        A ← A ∪ {e}
```

KRUSKAL IS THE GENERIC GREEDY

Instantiate the generic greedy on the graphic matroid: sort the edges, and add e whenever $A \cup \{e\}$ is still a forest.

"Keeps A a forest" is exactly "endpoints in different sets of the partition" — this *is* Kruskal's algorithm.

> **Second proof** The greedy theorem for matroids applies: Kruskal returns a minimum-weight maximal admissible set — a minimum spanning tree.

And the same proof covers every matroid, not just graphs.

UNION-FIND

IMPLEMENTING KRUSKAL

How do we implement Kruskal's algorithm? Each round needs to do three tasks:

1. identify the i^{th} lightest edge;
2. determine if its endpoints are in the same set of the partition;
3. (potentially) merge the sets containing its endpoints.

> **TASK 1** Sort the edges by weight! Time $O(|E| \log |E|)$ — the bottleneck step of the whole algorithm.

Tasks 2 and 3 leave graphs behind entirely: maintain a partition of $\{0, \ldots, n-1\}$ under *same-set queries* and *merges*. We need a new data structure!

THE UNION-FIND DATA STRUCTURE

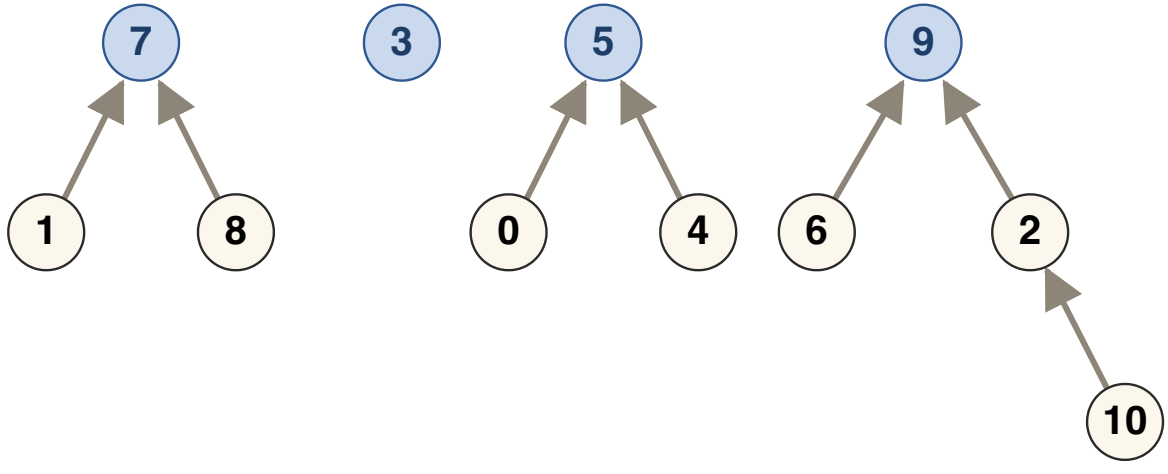
Also called the *disjoint-set* data structure. It maintains a partition and supports two operations:

> **FIND** `int find(int p)` — returns the *set identifier* of element p. Then p and q lie in the same set if and only if $\texttt{find}(p) == \texttt{find}(q)$. (Task 2.)

> **UNION** `void union(int p, int q)` — merges the set containing p with the set containing q. (Task 3.)

Example: partition $\{1, 7, 8\}$, $\{3\}$, $\{0, 4, 5\}$, $\{2, 6, 9, 10\}$. After `union(3,5)` the sets $\{3\}$ and $\{0, 4, 5\}$ become $\{0, 3, 4, 5\}$.

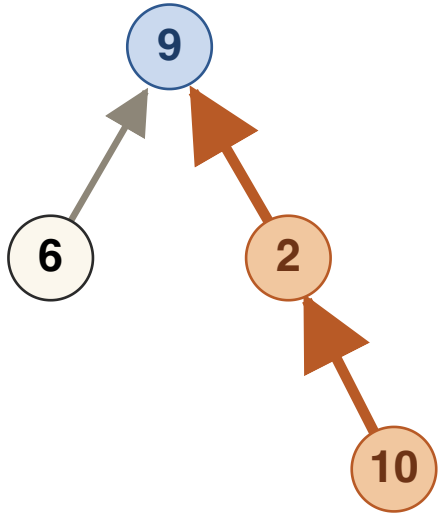
HIGH-LEVEL IDEA: EACH SET IS A TREE



The partition $\{1, 7, 8\}, \{3\}, \{0, 4, 5\}, \{2, 6, 9, 10\}$ as a forest. Arrows point from child to parent; roots are blue.

- Each set is represented by a tree of its elements, stored as a `parent` array.
- The **identifier of a set is the number at the root** of its tree.

FIND



`find(10)`: follow parents $10 \to 2 \to 9$, return 9.

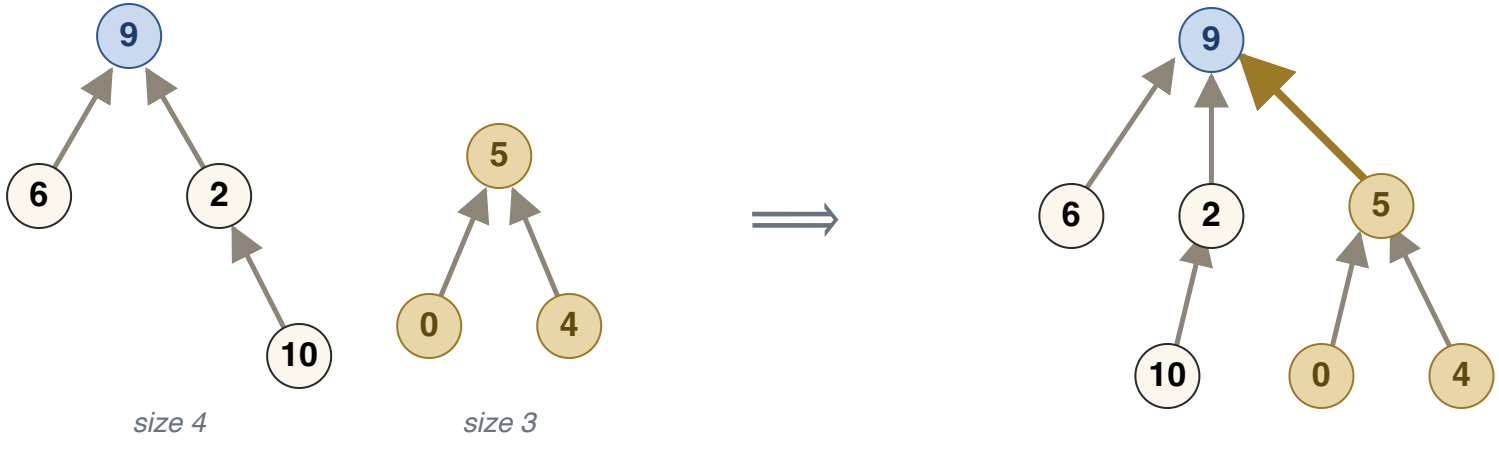
`find(p)`: trace our way up the tree from p to the root, and return the root.

The time taken is proportional to the **height of the tree**.

So the game is: keep the trees shallow.

UNION BY SIZE

`union(p,q)`: find the two roots; if they differ, link one root to the other.



`union(4,2)`: the roots are 5 and 9. The smaller tree (tan) hangs under 9.

> **RULE** The root of the **larger** tree becomes the parent of the **smaller** tree's root.

Cost: two finds plus one pointer update — again proportional to tree height.

WHY TREES STAY SHALLOW

CLAIM With union by size, the height of every tree is $O(\log n)$.

- A node's depth only increases when its root is linked under another root — and that happens only when its tree is merged with one *at least as large*.
- So each time an element gets one level deeper, the size of its tree at least **doubles**.
- Sizes cannot exceed n, so depth $\leq \log_2 n$.

CONSEQUENCE Simple union-find: `find` and `union` both run in time $O(\log n)$.

UNION-FIND: CODE

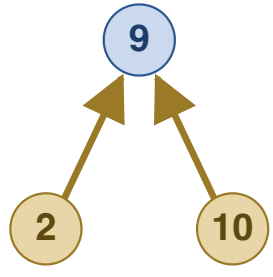
```
struct UnionFind:
    parent[i] ← i  for all i          // each element its own root
    size[i]   ← 1  for all i

    function find(p):
        while parent[p] ≠ p:            // walk up to the root
            p ← parent[p]
        return p

    function union(p, q):
        rp ← find(p);  rq ← find(q)
        if rp == rq: return             // already in the same set
        if size[rp] < size[rq]: swap(rp, rq)
        parent[rq] ← rp                 // smaller root under larger root
        size[rp] ← size[rp] + size[rq]
```

PATH COMPRESSION

Before `find(10)`



After: every visited node points directly at the root

Path compression: after a `find` walks to the root, re-point every node on the path directly at the root — flattening the tree for all future operations.

PATH COMPRESSION

THEOREM With path compression, any sequence of m union and find operations takes time $O(m \cdot \alpha(n))$, where α is the **inverse Ackermann function**.

$\alpha(n)$ is non-constant but extremely slow growing: $\alpha\left(2^{2^{2^{2^{16}}}}\right) \leq 5$. Note this is an *amortised* bound — not every single operation is guaranteed to take time $\alpha(n)$.

ASIDE: HOW FAST CAN MST BE?

The advanced union-find shows the bottleneck in Kruskal's algorithm is *sorting the edges*, not maintaining the partition.

- Chazelle gave an MST algorithm running in time $O(|E| \cdot \alpha(n))$ — "A Minimum Spanning Tree Algorithm with Inverse-Ackermann Type Complexity".
- It is still open whether MST can be solved in time $O(|E|)$ deterministically.

KRUSKAL WITH UNION-FIND

```
// Kruskal's algorithm with union-find
sort edges by weight, smallest first          // O(E log E)
UnionFind uf(n)
A ← ∅
for (u, v) in sorted edges:                   // at most E rounds
    if uf.find(u) ≠ uf.find(v):               // task 2: O(log n)
        A ← A ∪ {u, v}
        uf.union(u, v)                        // task 3: O(log n)
return A
```

KRUSKAL: RUNNING TIME

STEP	COST
sort edges	$O(\lvert E\rvert \log \lvert E\rvert)$
$\leq 2\lvert E\rvert$ finds + $\leq n-1$ unions	$O(\lvert E\rvert \log n)$
total	$O(\lvert E\rvert \log \lvert E\rvert) = O(\lvert E\rvert \log n)$

Since $\lvert E\rvert \leq n^2, \log \lvert E\rvert \leq 2 \log n$.

SUMMARY

- **Fact 2 (cut rule)**: the MST contains the minimum-weight edge of every cut — proved by a cycle-swap exchange argument.
- **Kruskal**: sort edges; add each edge whose endpoints lie in different components — safe by the cut rule.
- Second proof: forests form the **graphic matroid** (P0, P1, P2), and Kruskal is the generic greedy on it.
- **Union-find** maintains the components: $O(\log n)$ by union by size, amortised $O(\alpha(n))$ with path compression.
- Total: $O(|E| \log |E|)$, dominated by sorting.